

First-Order Theories

Formalization works as “an early-warning system” when things are getting contorted.

— Edsger W. Dijkstra
EWD764: Repaying Our Debts, 1980

When reasoning in particular application domains such as software or hardware, one often has particular structures in mind. For example, programs manipulate numbers, lists, and arrays. **First-order theories** formalize these structures to enable reasoning about them. This chapter introduces first-order theories in general and then focuses on theories useful in verification and related tasks. These theories include a theory of equality, of integers, of rationals and reals, of recursive data structures, and of arrays.

There is another reason to study first-order theories. While validity in FOL is undecidable, validity in particular theories or fragments of theories is sometimes decidable. Many of the theories studied in this chapter have important fragments for which validity is efficiently decidable. For each theory, we identify the decidable and efficiently decidable fragments, which we summarize in Section 3.7. Part II studies decision procedures for the decidable fragments.

3.1 First-Order Theories

A first-order **theory** T is defined by the following components.

1. Its **signature** Σ is a set of constant, function, and predicate symbols.
2. Its set of **axioms** $\mathcal{A}$ is a set of closed FOL formulae in which only constant, function, and predicate symbols of Σ appear.

A Σ -**formula** is constructed from constant, function, and predicate symbols of Σ , as well as variables, logical connectives, and quantifiers. As usual, the symbols of Σ are just symbols without prior meaning. The axioms $\mathcal{A}$ provide their meaning.

A Σ -formula F is **valid in the theory** T , or **T -valid**, if every interpretation I that satisfies the axioms of T ,

$$I \models A \quad \text{for every } A \in \mathcal{A}, \quad (3.1)$$

also satisfies F : $I \models F$. For this reason, we write

$$T \models F$$

to mean that F is T -valid. Formally, the theory T consists of all (closed) formulae that are T -valid. We call an interpretation satisfying (3.1) a **T -interpretation**.

A Σ -formula F is **satisfiable in** T , or **T -satisfiable**, if there is a T -interpretation I that satisfies F .

A theory T is **complete** if for every closed Σ -formula F , $T \models F$ or $T \models \neg F$. A theory is **consistent** if there is at least one T -interpretation. In particular, in a consistent theory T , there does not exist a Σ -formula F such that both $T \models F$ and $T \models \neg F$. (Otherwise, by the semantics of conjunction, $T \models F \wedge \neg F$ and thus $T \models \perp$; but $\perp$ is not satisfied by any interpretation.)

Concepts from general FOL validity carry over to first-order theories in the natural way. For example, two formulae F_1 and F_2 are **equivalent in** T , or **T -equivalent**, if $T \models F_1 \leftrightarrow F_2$: for every T -interpretation I , $I \models F_1$ iff $I \models F_2$.

A **fragment** of a theory is a syntactically-restricted subset of formulae of the theory. For example, the **quantifier-free fragment** of a theory T is the set of formulae without quantifiers that are valid in T . Recall our convention that non-closed formula F is valid iff its universal closure is valid. Technically, the “quantifier-free fragment” of T actually consists of valid formulae in which all variables are universally quantified. However, the term “quantifier-free fragment” is the common and accepted name for this fragment. Subsequent chapters show that the quantifier-free fragments of theories are of great practical and theoretical importance.

A theory T is **decidable** if $T \models F$ is decidable for every Σ -formula F . That is, there is an algorithm that always terminates with “yes” if F is T -valid or with “no” if F is T -invalid. A fragment of T is decidable if $T \models F$ is decidable for every Σ -formula F that obeys the fragment’s syntactic restrictions.

The union $T_1 \cup T_2$ of two theories T_1 and T_2 has signature $\Sigma_1 \cup \Sigma_2$ and axioms $\mathcal{A}_1 \cup \mathcal{A}_2$. Clearly, a $(T_1 \cup T_2)$ -interpretation is both a T_1 -interpretation and a T_2 -interpretation since it satisfies the axioms of both T_1 and T_2 . Hence, a formula that is T_1 -valid or T_2 -valid is $(T_1 \cup T_2)$ -valid, while a formula that is $(T_1 \cup T_2)$ -satisfiable is both T_1 -satisfiable and T_2 -satisfiable.

Because FOL (the “empty” theory, or the theory without axioms) is undecidable in general, we must turn to theories and fragments of theories for the possibility of fully automated reasoning. While many interesting theories are undecidable, there are several important theories and fragments of theories that are decidable. These theories and fragments are the main subject of Part

II of this book. We introduce them in the following sections. In Section 3.7, we summarize the decidability and complexity results for these theories and fragments.

3.2 Equality

The theory of equality T_E is the simplest first-order theory. Its signature

$$\Sigma_E : \{=, a, b, c, \dots, f, g, h, \dots, p, q, r, \dots\}$$

consists of

- $=$ (equality), a binary predicate;
- and all constant, function, and predicate symbols.

Equality $=$ is an **interpreted** predicate symbol: its meaning is defined via the axioms of T_E . The other constant, function, and predicate symbols are uninterpreted except as they relate to equality. The axioms of T_E are the following:

1. $\forall x. x = x$ (reflexivity)
2. $\forall x, y. x = y \rightarrow y = x$ (symmetry)
3. $\forall x, y, z. x = y \wedge y = z \rightarrow x = z$ (transitivity)
4. for each positive integer n and n -ary function symbol f ,

$$\forall \bar{x}, \bar{y}. \left(\bigwedge_{i=1}^n x_i = y_i \right) \rightarrow f(\bar{x}) = f(\bar{y}) \quad (\text{function congruence})$$

5. for each positive integer n and n -ary predicate symbol p ,

$$\forall \bar{x}, \bar{y}. \left(\bigwedge_{i=1}^n x_i = y_i \right) \rightarrow (p(\bar{x}) \leftrightarrow p(\bar{y})) \quad (\text{predicate congruence})$$

The notation $\bar{x}$ stands for the list of variables $x_1, \dots, x_n$. Axioms (function congruence) and (predicate congruence) are actually axiom schemata. An **axiom schema** stands for a set of axioms, each an instantiation of the parameters (f and p in (function congruence) and (predicate congruence), respectively). For example, for binary function symbol f_2 , (function congruence) instantiates to the following axiom:

$$\forall x_1, x_2, y_1, y_2. x_1 = y_1 \wedge x_2 = y_2 \rightarrow f_2(x_1, x_2) = f_2(y_1, y_2).$$

The first three axioms state that $=$ is an **equivalence relation**: it is a binary predicate that obeys reflexivity, symmetry, and transitivity. The final two axiom schemata formalize our intuition for the behavior of functions and predicates under equality. A function (predicate) always evaluates to the same

value (truth value) for a given set of argument values. They assert that $=$ is a **congruence relation**.

T_E is just as undecidable as full FOL because it allows all constant, function, and predicate symbols. In particular, any FOL formula F can be encoded as a Σ_E -formula F' simply by replacing occurrences of the symbol $=$ with a fresh symbol. Since $=$ does not occur in this transformed formula F' , the axioms of T_E are irrelevant; hence, F' is T_E -satisfiable iff F' is first-order satisfiable.

However, the quantifier-free fragment of T_E is both interesting and efficiently decidable, as we show in Chapter 9.

Example 3.1. Without quantifiers, free variables and constants play the same role. In the formula

$$F : a = b \wedge b = c \rightarrow g(f(a), b) = g(f(c), a) ,$$

a , b , and c are constants, while in

$$F' : x = y \wedge y = z \rightarrow g(f(x), y) = g(f(z), x) ,$$

x , y , and z are free variables. F is T_E -valid iff F' is T_E -valid; F is T_E -satisfiable iff F' is T_E -satisfiable. ■

It is often useful to reason about the T -satisfiability or T -validity of a Σ -formula F in a structured but informal way. We show how to use the semantic argument method with T_E .

Example 3.2. To prove that

$$F : a = b \wedge b = c \rightarrow g(f(a), b) = g(f(c), a)$$

is T_E -valid, assume otherwise: there exists a T_E -interpretation I such that $I \not\models F$:

- | | | |
|-----|---|----------------------------|
| 1. | $I \not\models F$ | assumption |
| 2. | $I \models a = b \wedge b = c$ | 1, $\rightarrow$ |
| 3. | $I \not\models g(f(a), b) = g(f(c), a)$ | 1, $\rightarrow$ |
| 4. | $I \models a = b$ | 2, $\wedge$ |
| 5. | $I \models b = c$ | 2, $\wedge$ |
| 6. | $I \models a = c$ | 4, 5, (transitivity) |
| 7. | $I \models f(a) = f(c)$ | 6, (function congruence) |
| 8. | $I \models b = a$ | 4, (symmetry) |
| 9. | $I \models g(f(a), b) = g(f(c), a)$ | 7, 8 (function congruence) |
| 10. | $I \models \perp$ | 3, 9 |

Our assumption is apparently false: F is T_E -valid. ■

3.3 Natural Numbers and Integers

Arithmetic involving the addition and multiplication of the natural numbers $\mathbb{N} = \{0, 1, 2, \dots\}$ is perhaps the oldest of mathematical theories. In this section we describe three theories of arithmetic. **Peano arithmetic** allows addition and multiplication over natural numbers, while **Presburger arithmetic** is restricted to addition over natural numbers. The final theory, the **theory of integers**, is convenient for automated reasoning but is no more expressive than Presburger arithmetic.

3.3.1 Peano Arithmetic

The **theory of Peano arithmetic** T_{PA} , or **first-order arithmetic**, has signature

$$\Sigma_{PA} : \{0, 1, +, \cdot, =\},$$

where

- 0 and 1 are constants;
- + (addition) and $\cdot$ (multiplication) are binary functions;
- and = (equality) is a binary predicate.

Its axioms are the following:

1. $\forall x. \neg(x + 1 = 0)$ (zero)
2. $\forall x, y. x + 1 = y + 1 \rightarrow x = y$ (successor)
3. $F[0] \wedge (\forall x. F[x] \rightarrow F[x + 1]) \rightarrow \forall x. F[x]$ (induction)
4. $\forall x. x + 0 = x$ (plus zero)
5. $\forall x, y. x + (y + 1) = (x + y) + 1$ (plus successor)
6. $\forall x. x \cdot 0 = 0$ (times zero)
7. $\forall x, y. x \cdot (y + 1) = x \cdot y + x$ (times successor)

These axioms concisely define addition, multiplication, and equality over natural numbers. Informally, axioms (zero), (plus zero), and (times zero) define 0 as we understand it: it is the minimal element of the natural numbers; it is the identity for addition ($x + 0 = x$); and under multiplication, it maps any number to 0 ($x \cdot 0 = 0$). Axioms (zero), (successor), (plus zero), and (plus successor) define addition. Axioms (times zero) and (times successor) define multiplication: in particular, (times successor) defines multiplication in terms of addition.

(induction) is an axiom schema: it stands for the set of axioms obtained by substituting for F each Σ_{PA} -formula that has precisely one free variable. It asserts that every T_{PA} -interpretation I obeys induction: if I satisfies $F[0]$ and $\forall x. F[x] \rightarrow F[x + 1]$, then I also satisfies $\forall x. F[x]$.

For convenience, we usually do not write the “ $\cdot$ ” for multiplication. For example, we write xy rather than $x \cdot y$.

The **intended interpretations** of T_{PA} have domain $\mathbb{N}$ and assignments α_I defining 0, 1, +, ·, and = as we understand them in everyday arithmetic. In particular,

- $\alpha_I[0]$ is $0_{\mathbb{N}}$: α_I maps the symbols “0” to $0_{\mathbb{N}} \in \mathbb{N}$;
- $\alpha_I[1]$ is $1_{\mathbb{N}}$: α_I maps the symbols “1” to $1_{\mathbb{N}} \in \mathbb{N}$;
- $\alpha_I[+]$ is $+_{\mathbb{N}}$, addition over $\mathbb{N}$;
- $\alpha_I[\cdot]$ is $\cdot_{\mathbb{N}}$, multiplication over $\mathbb{N}$;
- $\alpha_I[=]$ is $=_{\mathbb{N}}$, equality over $\mathbb{N}$.

Example 3.3. The formula $3x + 5 = 2y$ can be written using the signature Σ_{PA} as

$$x + x + x + 1 + 1 + 1 + 1 + 1 = y + y$$

or as

$$(1 + 1 + 1) \cdot x + 1 + 1 + 1 + 1 + 1 = (1 + 1) \cdot y .$$

In practice, we use the abbreviated notation $3x + 5 = 2y$. ■

Example 3.4. Rather than augmenting T_{PA} with axioms defining inequality $>$, we can transform formulae with inequality into formulae over the restricted signature Σ_{PA} . Write

$$3x + 5 > 2y \quad \text{as} \quad \exists z. z \neq 0 \wedge 3x + 5 = 2y + z ,$$

where $z \neq 0$ abbreviates $\neg(z = 0)$. The latter formula is a Σ_{PA} -formula. Weak inequality can be similarly transformed. Write

$$3x + 5 \geq 2y \quad \text{as} \quad \exists z. 3x + 5 = 2y + z .$$

■

Example 3.5. The Σ_{PA} -formula

$$\exists x, y, z. x \neq 0 \wedge y \neq 0 \wedge z \neq 0 \wedge xx + yy = zz$$

is T_{PA} -valid. It asserts that there exists a triple of positive integers fulfilling the Pythagorean Theorem. The formula

$$\exists x, y, z. x \neq 0 \wedge y \neq 0 \wedge z \neq 0 \wedge xxx + yyy = zzz$$

is the cubic analogue. For constant n , let x^n represent n multiplications of x ; then every formula of the set

$$\{\forall x, y, z. x \neq 0 \wedge y \neq 0 \wedge z \neq 0 \rightarrow x^n + y^n \neq z^n : n > 2 \wedge n \in \mathbb{Z}\}$$

is T_{PA} -valid, as claimed by Fermat’s Last Theorem and proved by Andrew Wiles in 1994. ■

Remark 3.6. Gödel's first incompleteness theorem (see **Bibliographic Remarks** of Chapter 2) implies that Peano arithmetic T_{PA} does not capture true arithmetic: there exist closed Σ_{PA} -formulae representing valid propositions of number theory that are T_{PA} -invalid. Gödel's proof constructs such a formula: it encodes the assertion that the formula itself cannot be proved. Now, either this formula can be proved from the axioms of T_{PA} (contradicting itself so that T_{PA} is inconsistent) or it cannot be proved (so that T_{PA} is incomplete).

Satisfiability and validity in T_{PA} is undecidable. Therefore, we turn to a more restricted theory of arithmetic that does not allow multiplication.

3.3.2 Presburger Arithmetic

The **theory of Presburger arithmetic** $T_{\mathbb{N}}$ has signature

$$\Sigma_{\mathbb{N}} : \{0, 1, +, =\},$$

where

- 0 and 1 are constants;
- + (addition) is a binary function;
- and = (equality) is a binary predicate.

Its axioms are a subset of the axioms of T_{PA} :

1. $\forall x. \neg(x + 1 = 0)$ (zero)
2. $\forall x, y. x + 1 = y + 1 \rightarrow x = y$ (successor)
3. $F[0] \wedge (\forall x. F[x] \rightarrow F[x + 1]) \rightarrow \forall x. F[x]$ (induction)
4. $\forall x. x + 0 = x$ (plus zero)
5. $\forall x, y. x + (y + 1) = (x + y) + 1$ (plus successor)

Again, (induction) is an axiom schema standing for the set of axioms obtained by replacing F with each $\Sigma_{\mathbb{N}}$ -formula that has precisely one free variable.

The intended interpretations of $T_{\mathbb{N}}$ have domain $\mathbb{N}$ and are such that

- $\alpha_I[0]$ is $0_{\mathbb{N}} \in \mathbb{N}$;
- $\alpha_I[1]$ is $1_{\mathbb{N}} \in \mathbb{N}$;
- $\alpha_I[+]$ is $+_{\mathbb{N}}$, addition over $\mathbb{N}$;
- $\alpha_I[=]$ is $=_{\mathbb{N}}$, equality over $\mathbb{N}$.

How does one reason about all integers, $\mathbb{Z} = \{\dots, -2, -1, 0, 1, 2, \dots\}$? Such formulae can be encoded as $\Sigma_{\mathbb{N}}$ -formulae.

Example 3.7. Consider the formula

$$F_0 : \forall w, x. \exists y, z. x + 2y - z - 13 > -3w + 5,$$

where $-$ is meant to be interpreted as standard subtraction, and w, x, y , and z are intended to range over $\mathbb{Z}$. The formula

$$F_1 : \quad \forall w_p, w_n, x_p, x_n. \exists y_p, y_n, z_p, z_n. \\ (x_p - x_n) + 2(y_p - y_n) - (z_p - z_n) - 13 > -3(w_p - w_n) + 5$$

introduces two variables, v_p and v_n , for each variable v of F_0 . While each of v_p and v_n can only range over $\mathbb{N}$, $v_p - v_n$ should range over the integers. But how is $-$ interpreted? Moving negated terms to the other side of the inequality eliminates $-$:

$$F_2 : \quad \forall w_p, w_n, x_p, x_n. \exists y_p, y_n, z_p, z_n. \\ x_p + 2y_p + z_n + 3w_p > x_n + 2y_n + z_p + 13 + 3w_n + 5.$$

The final transformation eliminates constant coefficients and strict inequality:

$$F_3 : \quad \forall w_p, w_n, x_p, x_n. \exists y_p, y_n, z_p, z_n. \exists u. \\ \neg(u = 0) \wedge \\ x_p + y_p + y_p + z_n + w_p + w_p + w_p \\ = x_n + y_n + y_n + z_p + w_n + w_n + w_n + u \\ + 1 + 1 + 1 + 1 + 1 + 1 + 1 + 1 + 1 + 1 \\ + 1 + 1 + 1 + 1 + 1 + 1 + 1 + 1 + 1 + 1.$$

■

Presburger showed in 1929 that $T_{\mathbb{N}}$ is decidable. Therefore, the “theory of (negative and positive) integers” that we loosely constructed above is also decidable via the syntactic rewriting of formulae into $\Sigma_{\mathbb{N}}$ -formulae. Rather than using this cumbersome rewriting, however, we next study a theory of integers.

3.3.3 Theory of Integers

The **theory of integers** $T_{\mathbb{Z}}$ has signature

$$\Sigma_{\mathbb{Z}} : \{ \dots, -2, -1, 0, 1, 2, \dots, -3\cdot, -2\cdot, 2\cdot, 3\cdot, \dots, +, -, =, > \},$$

where

- $\dots, -2, -1, 0, 1, 2, \dots$ are constants, intended to be assigned the obvious corresponding values in the intended domain of integers $\mathbb{Z}$;
- $\dots, -3\cdot, -2\cdot, 2\cdot, 3\cdot, \dots$ are unary functions, intended to represent constant coefficients (*e.g.*, $2 \cdot x$, abbreviated $2x$);
- $+$ and $-$ are binary functions, intended to represent the obvious corresponding functions over $\mathbb{Z}$;
- $=$ and $>$ are binary predicates, intended to represent the obvious corresponding predicates over $\mathbb{Z}$.

Since Example 3.7 shows that $\Sigma_{\mathbb{Z}}$ -formulae can be reduced to $\Sigma_{\mathbb{N}}$ -formulae, we do not axiomatize $T_{\mathbb{Z}}$. $T_{\mathbb{Z}}$ is merely a convenient representation for reasoning about addition over all integers.

The intended interpretations of $T_{\mathbb{Z}}$ have domain $\mathbb{Z}$ and are such that α_I assigns the obvious values, functions, and predicates to the constant, function, and predicate symbols of $\Sigma_{\mathbb{Z}}$.

In Chapter 7, we discuss Cooper's decision procedure for deciding $T_{\mathbb{Z}}$ -validity, while in Chapter 8, we discuss decision procedures for the quantifier-free fragment of $T_{\mathbb{Z}}$. These procedures decide $T_{\mathbb{N}}$ -validity as well: the following example illustrates that $\Sigma_{\mathbb{N}}$ -formulae can be reduced to $\Sigma_{\mathbb{Z}}$ -formulae.

Example 3.8. To decide the $T_{\mathbb{N}}$ -validity of

$$\forall x. \exists y. x = y + 1 ,$$

decide the $T_{\mathbb{Z}}$ -validity of

$$\forall x. x \geq 0 \rightarrow \exists y. y \geq 0 \wedge x = y + 1 ,$$

where $t_1 \geq t_2$ expands to $t_1 = t_2 \vee t_1 > t_2$. ■

We prove the validity of several $\Sigma_{\mathbb{Z}}$ -formulae using the semantic argument method. Our application of the semantic argument method in this context is informal; it is intended to allow us to argue intuitively about validity until Chapter 7.

Example 3.9. To prove that the $\Sigma_{\mathbb{Z}}$ -formula

$$F : \forall x, y, z. x > z \wedge y \geq 0 \rightarrow x + y > z .$$

is $T_{\mathbb{Z}}$ -valid, assume otherwise: there is a $T_{\mathbb{Z}}$ -interpretation I such that $I \not\models F$:

- | | | |
|----|---|------------------|
| 1. | I $\not\models F$ | assumption |
| 2. | $I : I \triangleleft \{x \mapsto v_1\} \triangleleft \{y \mapsto v_2\} \triangleleft \{z \mapsto v_3\}$ | |
| | $\not\models x > z \wedge y \geq 0 \rightarrow x + y > z$ | 1, $\forall$ |
| 3. | $I_1 \models x > z \wedge y \geq 0$ | 2, $\rightarrow$ |
| 4. | $I_1 \not\models x + y > z$ | 2, $\rightarrow$ |
| 5. | $I_1 \models \neg(x + y > z)$ | 4, $\neg$ |

We derive a contradiction by collecting formulae from lines 3 and 5, applying the variant interpretation I_1 , and querying the theory $T_{\mathbb{Z}}$: are there integers v_1, v_2, v_3 such that

$$v_1 > v_3 \wedge v_2 \geq 0 \wedge \neg(v_1 + v_2 > v_3) ?$$

No, for $v_1 > v_3 \wedge v_2 \geq 0$ implies $v_1 + v_2 > v_3$. We summarize this reasoning in $T_{\mathbb{Z}}$ with the line

$$6. \quad I_1 \models \perp \quad 3, 5, T_{\mathbb{Z}}$$

Therefore, F is $T_{\mathbb{Z}}$ -valid. ■

Arguing the validity of arithmetic formulae at the level of axioms is tedious. Therefore, unlike in Example 3.2 in which the theory-specific reasoning is incorporated into the semantic argument method by applying and stating specific axioms of T_E , the semantic argument method for $T_{\mathbb{Z}}$ handles the “logical” aspects of the structured reasoning, while a separate informal argument reasons about the theory-specific elements.

Example 3.10. To prove that the $\Sigma_{\mathbb{Z}}$ -formula

$$F : \forall x, y. x > 0 \wedge (x = 2y \vee x = 2y + 1) \rightarrow x - y > 0$$

is $T_{\mathbb{Z}}$ -valid, assume otherwise: there is a $T_{\mathbb{Z}}$ -interpretation I such that $I \not\models F$:

- | | | |
|----|--|------------------|
| 1. | $I \not\models F$ | assumption |
| 2. | $I_1 : I \triangleleft \{x \mapsto v_1\} \triangleleft \{y \mapsto v_2\}$
$\not\models x > 0 \wedge (x = 2y \vee x = 2y + 1) \rightarrow x - y > 0$ | |
| | | 1, $\forall$ |
| 3. | $I_1 \models x > 0 \wedge (x = 2y \vee x = 2y + 1)$ | 2, $\rightarrow$ |
| 4. | $I_1 \models x > 0$ | 3, $\wedge$ |
| 5. | $I_1 \models x = 2y \vee x = 2y + 1$ | 3, $\wedge$ |
| 6. | $I_1 \not\models x - y > 0$ | 2, $\rightarrow$ |
| 7. | $I_1 \models \neg(x - y > 0)$ | 6, $\neg$ |

There are two cases to consider. In the first case,

$$8a. \quad I_1 \models x = 2y \quad 5, \vee$$

We collect the formulae of lines 4, 7, and 8a, apply the variant interpretation I_1 , and query the theory $T_{\mathbb{Z}}$: are there integers v_1, v_2 such that

$$v_1 > 0 \wedge v_1 = 2v_2 \wedge \neg(v_1 - v_2 > 0) ?$$

No, for substituting $v_1 = 2v_2$ throughout produces

$$2v_2 > 0 \wedge \neg(2v_2 - v_2 > 0) ,$$

which simplifies to

$$v_2 > 0 \wedge \neg(v_2 > 0) ,$$

a contradiction. This reasoning is summarized by

$$9a. \quad I_1 \models \perp \quad 4, 7, 8a, T_{\mathbb{Z}}$$

In the second case,

$$8b. \quad I_1 \models x = 2y + 1 \quad 5, \vee$$

Considering lines 4, 7, and 8b, are there integers v_1, v_2 such that

$$v_1 > 0 \wedge v_1 = 2v_2 + 1 \wedge \neg(v_1 - v_2 > 0) ?$$

No, for substituting $v_1 = 2v_2 + 1$ throughout produces

$$2v_2 + 1 > 0 \wedge \neg(2v_2 + 1 - v_2 > 0) ,$$

which simplifies to

$$2v_2 + 1 > 0 \wedge \neg(v_2 + 1 > 0) .$$

The first literal holds only when $v_2 > -1$, while the second holds only when $v_2 \leq -1$, a contradiction. This reasoning is summarized by

$$9b. \quad I_1 \models \perp \quad 4, 7, 8b, T_{\mathbb{Z}}$$

Thus, F is $T_{\mathbb{Z}}$ -valid. ■

3.4 Rationals and Reals

Almost as old as arithmetic on integers is arithmetic on the rational numbers $\mathbb{Q}$ and (not quite as old) on the real numbers $\mathbb{R}$. In this section, we describe two theories of real arithmetic. The latter theory can also be seen as a theory of rational arithmetic.

The first theory is the **theory of reals**, involving addition and multiplication over $\mathbb{R}$; it is also known as **elementary algebra**. The term “elementary” refers to the restriction that variables range only over domain elements (as in all first-order theories), not over sets or functions of domain elements. Most junior high students are familiar with elementary algebra. As a first-order theory, elementary algebra is of course more complex since formulae are constructed with logical connectives and quantifiers.

The second theory is the theory of addition over $\mathbb{R}$ or $\mathbb{Q}$. Interpretations with domains of $\mathbb{R}$ are indistinguishable from interpretations with domains of $\mathbb{Q}$, as we discuss below. For this reason, we call this second theory the **theory of rationals**.

Example 3.11. Let us distinguish informally between the theories of integers (with only addition), reals (with addition and multiplication), and rationals (with only addition).

In the theory of integers,

$$F : \exists x. 2x = 7$$

is $T_{\mathbb{Z}}$ -invalid. However, assigning to x the rational number $\frac{7}{2}$ satisfies $2x = 7$, so F should be satisfiable in the theory of rationals. Moreover, $\frac{7}{2}$ is also a real number, so F should also be satisfiable in the theory of reals.

The theory of reals includes multiplication, allowing a formula like

$$G : \exists x. x^2 = 2$$

to be expressed, where x^2 abbreviates $x \cdot x$. G should be valid in the theory of reals because assigning to x the real number $\sqrt{2}$ satisfies $x^2 = 2$. $\sqrt{2}$ is irrational. ■

3.4.1 Theory of Reals

The **theory of reals** $T_{\mathbb{R}}$, or **elementary algebra**, has signature

$$\Sigma_{\mathbb{R}} : \{0, 1, +, -, \cdot, =, \geq\},$$

where

- 0 and 1 are constants;
- + (addition) and $\cdot$ (multiplication) are binary functions;
- − (negation) is a unary function;
- and = (equality) and $\geq$ (weak inequality) are binary predicates.

$T_{\mathbb{R}}$ has the most complex axiomatization of the theories that we study. We group axioms by their mathematical content.

First are the axioms of an **abelian group**. An abelian group is a structure with additive identity 0, associative and commutative addition +, additive inverse −, and equality =. The qualifier “abelian” simply means that addition is commutative. The axioms are the following:

1. $\forall x, y, z. (x + y) + z = x + (y + z)$ (+ associativity)
2. $\forall x. x + 0 = x$ (+ identity)
3. $\forall x. x + (-x) = 0$ (+ inverse)
4. $\forall x, y. x + y = y + x$ (+ commutativity)

The first three axioms are the axioms of a **group**.

Second are the additional axioms of a **ring**. A ring is an abelian group with a multiplicative identity 1 and associative multiplication $\cdot$ that distributes over addition. For convenience, we usually shorten $x \cdot y$ to xy .

1. $\forall x, y, z. (xy)z = x(yz)$ ($\cdot$ associativity)
2. $\forall x. x1 = x$ ($\cdot$ left identity)
3. $\forall x. 1x = x$ ($\cdot$ right identity)
4. $\forall x, y, z. x(y + z) = xy + xz$ (left distributivity)
5. $\forall x, y, z. (x + y)z = xz + yz$ (right distributivity)

Both left and right identity and distributivity axioms are required since $\cdot$ is not commutative (yet). It is made so in the next set of axioms.

Third are the additional axioms of a **field**. In a field, $\cdot$ is commutative; the additive and multiplicative identities are different; and the multiplicative inverse of a non-0 value exists (e.g., $\frac{1}{2}$ is the multiplicative inverse of 2).

1. $\forall x, y. xy = yx$ ($\cdot$ commutativity)

2. $0 \neq 1$ (separate identities)
3. $\forall x. x \neq 0 \rightarrow \exists y. xy = 1$ ($\cdot$ inverse)

The axiom ($\cdot$ commutativity) makes the ($\cdot$ right identity) and (right distributivity) axioms redundant.

Fourth are the additional axioms characterizing $\geq$ as a **total order**.

1. $\forall x, y. x \geq y \wedge y \geq x \rightarrow x = y$ (antisymmetry)
2. $\forall x, y, z. x \geq y \wedge y \geq z \rightarrow x \geq z$ (transitivity)
3. $\forall x, y. x \geq y \vee y \geq x$ (totality)

Finally are the additional axioms of a **real closed field**.

1. $\forall x, y, z. x \geq y \rightarrow x + z \geq y + z$ ($+$ ordered)
2. $\forall x, y. x \geq 0 \wedge y \geq 0 \rightarrow xy \geq 0$ ($\cdot$ ordered)
3. $\forall x. \exists y. x = y^2 \vee x = -y^2$ (square-root)
4. for each odd integer n ,

$$\forall \bar{x}. \exists y. y^n + x_1 y^{n-1} + \cdots + x_{n-1} y + x_n = 0 \quad (\text{at least one root})$$

We again abbreviate $x_1, \dots, x_n$ by $\bar{x}$. By y^n , we mean y multiplied by itself n times: $y \cdots y$. The axioms ($+$ ordered) and ($\cdot$ ordered) assert that every $T_{\mathbb{R}}$ -interpretation is an **ordered field**. The axiom (square-root) asserts the existence of the square-root of every value. The final axiom schema states that polynomials of odd degree have at least one root.

Putting all axioms together and pruning redundant axioms, we have:

1. $\forall x, y. x \geq y \wedge y \geq x \rightarrow x = y$ (antisymmetry)
2. $\forall x, y, z. x \geq y \wedge y \geq z \rightarrow x \geq z$ (transitivity)
3. $\forall x, y. x \geq y \vee y \geq x$ (totality)
4. $\forall x, y, z. (x + y) + z = x + (y + z)$ ($+$ associativity)
5. $\forall x. x + 0 = x$ ($+$ identity)
6. $\forall x. x + (-x) = 0$ ($+$ inverse)
7. $\forall x, y. x + y = y + x$ ($+$ commutativity)
8. $\forall x, y, z. x \geq y \rightarrow x + z \geq y + z$ ($+$ ordered)
9. $\forall x, y, z. (xy)z = x(yz)$ ($\cdot$ associativity)
10. $\forall x. 1x = x$ ($\cdot$ identity)
11. $\forall x. x \neq 0 \rightarrow \exists y. xy = 1$ ($\cdot$ inverse)
12. $\forall x, y. xy = yx$ ($\cdot$ commutativity)
13. $\forall x, y. x \geq 0 \wedge y \geq 0 \rightarrow xy \geq 0$ ($\cdot$ ordered)
14. $\forall x, y, z. x(y + z) = xy + xz$ (distributivity)
15. $0 \neq 1$ (separate identities)
16. $\forall x. \exists y. x = y^2 \vee -x = y^2$ (square-root)

17. for each odd integer n ,

$$\forall \bar{x}. \exists y. y^n + x_1 y^{n-1} + \cdots + x_{n-1} y + x_n = 0 \quad (\text{at least one root})$$

Example 3.12. The method of **quantifier elimination**, which we study in Chapter 7, eliminates quantifiers from a formula to produce an equivalent quantifier-free formula. If a formula F contains free variables, then a quantifier elimination procedure produces an equivalent quantifier-free formula F' such that $\text{free}(F') \subseteq \text{free}(F)$. For example, when is the formula

$$F : \exists x. ax^2 + bx + c = 0$$

satisfiable? That is, what are the conditions on a , b , and c such that a quadratic polynomial has a real root? Recall that the discriminant must be nonnegative:

$$F' : b^2 - 4ac \geq 0 .$$

F' is the quantifier-free formula that is $T_{\mathbb{R}}$ -equivalent to F . ■

Tarski proved that $T_{\mathbb{R}}$ was decidable in the 1930s, although the Second World War prevented his publishing the result until 1956. Collins proposed the more efficient technique of **cylindrical algebraic decomposition** (CAD) in 1975. Unfortunately, even the most efficient decision procedures for $T_{\mathbb{R}}$ have prohibitively high time complexity: CAD runs in time proportionate to $2^{2^{k|F|}}$, for some constant k and for $|F|$ the length of $\Sigma_{\mathbb{R}}$ -formula F .

3.4.2 Theory of Rationals

Given the high complexity of deciding $T_{\mathbb{R}}$ -validity (and the high intellectual complexity of Tarski's and subsequent decision procedures for $T_{\mathbb{R}}$), we turn to a simpler theory without multiplication, the **theory of rationals** $T_{\mathbb{Q}}$. It has signature

$$\Sigma_{\mathbb{Q}} : \{0, 1, +, -, =, \geq\} ,$$

where

- 0 and 1 are constants;
- $+$ (addition) is a binary function;
- $-$ (negation) is a unary function;
- and $=$ (equality) and $\geq$ (weak inequality) are binary predicates.

Its axioms are the following:

1. $\forall x, y. x \geq y \wedge y \geq x \rightarrow x = y$ (antisymmetry)
2. $\forall x, y, z. x \geq y \wedge y \geq z \rightarrow x \geq z$ (transitivity)
3. $\forall x, y. x \geq y \vee y \geq x$ (totality)

4. $\forall x, y, z. (x + y) + z = x + (y + z)$ (+ associativity)
5. $\forall x. x + 0 = x$ (+ identity)
6. $\forall x. x + (-x) = 0$ (+ inverse)
7. $\forall x, y. x + y = y + x$ (+ commutativity)
8. $\forall x, y, z. x \geq y \rightarrow x + z \geq y + z$ (+ ordered)

9. for each positive integer n ,

$$\forall x. nx = 0 \rightarrow x = 0 \quad (\text{torsion-free})$$

10. for each positive integer n ,

$$\forall x. \exists y. x = ny \quad (\text{divisible})$$

By nx we mean x added to itself n times: $x + \cdots + x$. The first eight axioms are a subset of the axioms of $T_{\mathbb{R}}$. They state that every $T_{\mathbb{Q}}$ -interpretation is an ordered abelian group. $\geq$ is a total order by the first three axioms. Identity 0, addition $+$, additive inverse $-$, and equality $=$ comprise an abelian group by the next four axioms. The eighth axiom asserts that the abelian group is ordered.

The axiom schema (**torsion-free**) states that only 0 can be added to itself to produce 0. The name “torsion-free” comes from the following mathematical context. In a group, the **order** of an element v is the integer n such that nv is the identity element 0: $nv = 0$. If no such n exists, then the element v has infinite order. A group is **torsion-free** if the only element with finite order is the identity 0.

Finally, the axiom schema (**divisible**) asserts that all elements of the domain D_I of a $T_{\mathbb{Q}}$ -interpretation I are divisible. That is, for every positive integer n , every element $v \in D_I$ is the sum of n of some other element $w \in D_I$.

Thus, every $T_{\mathbb{Q}}$ -interpretation is a divisible torsion-free abelian group. In particular, the rationals and reals with $+$, $-$, $=$, and $\geq$ are divisible torsion-free abelian groups. As $T_{\mathbb{Q}}$ -interpretations, the rationals and reals are **elementarily equivalent**: there does not exist a $\Sigma_{\mathbb{Q}}$ -formula that distinguishes between a real $T_{\mathbb{Q}}$ -interpretation (an interpretation with domain $\mathbb{R}$) and a rational $T_{\mathbb{Q}}$ -interpretation (an interpretation with domain $\mathbb{Q}$).

This characteristic makes sense, intuitively: no linear expression with only integer coefficients can capture, say, $\sqrt{2}$ without also being satisfied by some rational values. When junior high students solve linear algebra problems, they apply addition, subtraction, multiplication, and division; but they do not take roots.

In contrast, $T_{\mathbb{R}}$ is a theory of reals: the $\Sigma_{\mathbb{R}}$ -formula $x \cdot x = 2$ is only satisfied by $T_{\mathbb{R}}$ -interpretations I in which $\alpha_I[x] = -\sqrt{2}$ or $\alpha_I[x] = \sqrt{2}$.

Example 3.13. Strict inequality is simple to express in $T_{\mathbb{Q}}$. Write

$$\forall x, y. \exists z. x + y > z$$

as the $\Sigma_{\mathbb{Q}}$ -formula

$$\forall x, y. \exists z. \neg(x + y = z) \wedge x + y \geq z .$$

The situation is similar for $T_{\mathbb{R}}$. ■

Example 3.14. Rational coefficients are simple to express in $T_{\mathbb{Q}}$. Write

$$\frac{1}{2}x + \frac{2}{3}y \geq 4$$

as the $\Sigma_{\mathbb{Q}}$ -formula $3x + 4y \geq 24$. ■

For convenience, we sometimes write $x \leq y$ for $y \geq x$.

In Chapter 7, we study a procedure for eliminating quantifiers in the theory $T_{\mathbb{Q}}$. On closed formulae, this procedure decides validity. In Chapter 8, we study a decision procedure for the quantifier-free fragment of $T_{\mathbb{Q}}$, which is efficiently decidable.

3.5 Recursive Data Structures

The **theory of recursive data structures (RDS)** describes a set of data structures that are ubiquitous in programming. The most basic RDS is a non-recursive structure, like C’s `struct`, in which a single variable has multiple fields. Truly recursive RDSs include lists, stacks, and binary trees.

The theory of recursive data structures T_{RDS} formalizes the reasoning about such structures. It builds on the theory of equality T_{E} .

Theory of Lists

We first focus on the theory of LISP-like lists, T_{cons} , which has signature

$$\Sigma_{\text{cons}} : \{\text{cons}, \text{car}, \text{cdr}, \text{atom}, =\} ,$$

where

- `cons` is a binary function, called the constructor: `cons(a, b)` represents the list constructed by concatenating `a` to `b`;
- `car` is a unary function, called the left projector: `car(cons(a, b)) = a`;
- `cdr` is a unary function, called the right projector: `cdr(cons(a, b)) = b`;
- `atom` is a unary predicate: `atom(x)` is true iff `x` is a single-element list;
- and `=` (equality) is a binary predicate.

`car` and `cdr` are historical names abbreviating “contents of address register” and “contents of decrement register”, respectively. In the intended interpretations, atoms are individual elements, while lists are multiple elements assembled together via `cons`. For example, `cons(a, cons(b, c))` is a list of three

elements, while a for which $\text{atom}(a)$ holds is an atom. car and cdr are functions for accessing parts of lists. For example, $\text{car}(\text{cons}(a, \text{cons}(b, c)))$ returns the head a of the list; $\text{cdr}(\text{cons}(a, \text{cons}(b, c)))$ returns the tail $\text{cons}(b, c)$ of the list; and $\text{cdr}(\text{cdr}(\text{cons}(a, \text{cons}(b, c))))$ returns c .

The axioms of T_{cons} are the following:

1. the axioms of (reflexivity), (symmetry), and (transitivity) of T_E
2. instantiations of the (function congruence) axiom schema for cons , car , and cdr :

$$\forall x_1, x_2, y_1, y_2. x_1 = x_2 \wedge y_1 = y_2 \rightarrow \text{cons}(x_1, y_1) = \text{cons}(x_2, y_2)$$

$$\forall x, y. x = y \rightarrow \text{car}(x) = \text{car}(y)$$

$$\forall x, y. x = y \rightarrow \text{cdr}(x) = \text{cdr}(y)$$

3. an instantiation of the (predicate congruence) axiom schema for atom :

$$\forall x, y. x = y \rightarrow (\text{atom}(x) \leftrightarrow \text{atom}(y))$$

4. $\forall x, y. \text{car}(\text{cons}(x, y)) = x$ (left projection)
5. $\forall x, y. \text{cdr}(\text{cons}(x, y)) = y$ (right projection)
6. $\forall x. \neg \text{atom}(x) \rightarrow \text{cons}(\text{car}(x), \text{cdr}(x)) = x$ (construction)
7. $\forall x, y. \neg \text{atom}(\text{cons}(x, y))$ (atom)

The first three sets of axioms define $=$ to be a congruence relation for cons , car , cdr , and atom . The axioms (left projection) and (right projection) define the behavior of car and cdr on non-atom lists: car returns the first element of a cons structure, and cdr returns the second element. However, they do not specify the behavior of car and cdr on atoms. The (construction) axiom states that the cons of $\text{car}(x)$ and $\text{cdr}(x)$ is x itself, unless x is an atom. In other words, cons constructs structures, and car and cdr deconstructs them. Finally, the axiom (atom) asserts that a term with root function symbol cons is not an atom; it is a non-atomic list.

The congruence axioms for cons , car , and cdr assert an important property about lists: two lists are equal iff their components are equal. The forward direction — if two lists are equal, then their components are equal — is a consequence of the (function congruence) axioms for car and cdr . The backward direction is a consequence of the (function congruence) axiom for cons . This relationship between two structures and their components is sometimes called **extensionality**. We see it in arrays as well.

General Theory of RDS

T_{cons} is an instance of the general theory of recursive data structures T_{RDS} . Each RDS contributes the following to the signature:

- an n -ary constructor C ;

- n projection functions $\pi_1^C, \dots, \pi_n^C$;
- and one atom predicate atom_C .

Associated with each RDS is an instantiation of the following axiom schema:

1. the axioms of (reflexivity), (symmetry), and (transitivity) of T_E ;
2. instantiations of the (function congruence) axiom schema for constructor C and set of projectors $\pi_1^C, \dots, \pi_n^C$;
3. an instantiation of the (predicate congruence) axiom schema for atom_C ;
4. for each $i \in \{1, \dots, n\}$,

$$\forall x_1, \dots, x_n. \pi_i^C(C(x_1, \dots, x_n)) = x_i \quad (\text{projection})$$

$$5. \forall x. \neg \text{atom}_C(x) \rightarrow C(\pi_1^C(x), \dots, \pi_n^C(x)) = x \quad (\text{construction})$$

$$6. \forall x_1, \dots, x_n. \neg \text{atom}_C(C(x_1, \dots, x_n)) \quad (\text{atom})$$

The axioms of T_{cons} are an instantiation of this schema. We subsequently focus on T_{cons} for concreteness.

Theory of Acyclic Lists

A variation on this theory in which data structures are acyclic has been studied. Acyclicity makes sense for stacks, but not necessarily for lists and other data structures. Consider the theory of acyclic LISP-like lists, T_{cons}^+ . Its axioms include those of T_{cons} and the following axiom schema:

$$\begin{aligned} \forall x. \text{car}(x) &\neq x \\ \forall x. \text{cdr}(x) &\neq x \\ \forall x. \text{car}(\text{car}(x)) &\neq x \\ \forall x. \text{car}(\text{cdr}(x)) &\neq x \\ \forall x. \text{cdr}(\text{car}(x)) &\neq x \\ \dots \end{aligned}$$

T_{cons}^+ is decidable, but T_{cons} is not. However, the quantifier-free fragments of these theories are efficiently decidable.

Theory of Lists with Specified Atoms

The axioms of T_{cons} leave the behavior of car and cdr on atoms unspecified. Adding the axiom

$$\forall x. \text{atom}(x) \rightarrow \text{atom}(\text{car}(x)) \wedge \text{atom}(\text{cdr}(x))$$

to those of T_{cons} makes decidability of the resulting theory $T_{\text{cons}}^{\text{atom}}$ NP-complete.

Theory of Lists with Equality

In Chapter 9, we describe a decision procedure for satisfiability in the quantifier-free fragment of T_{cons} . The decision procedure is actually applicable to the quantifier-free fragment of a more expressive theory, $T_{\text{cons}}^=$, which is the combination of T_E and T_{cons} and thus includes uninterpreted constants, functions, and predicates. Thus, its signature is $\Sigma_E \cup \Sigma_{\text{cons}}$, and its axioms are the union of the axioms of T_E and T_{cons} . In Section 3.8 and Chapter 10, we discuss more general combinations of theories.

Example 3.15. To prove that the $\Sigma_{\text{cons}}^=$ -formula

$$F : \quad \begin{array}{l} \text{car}(a) = \text{car}(b) \wedge \text{cdr}(a) = \text{cdr}(b) \wedge \neg \text{atom}(a) \wedge \neg \text{atom}(b) \\ \rightarrow f(a) = f(b) \end{array}$$

is $T_{\text{cons}}^=$ -valid, assume otherwise: there exists a $T_{\text{cons}}^=$ -interpretation I such that $I \not\models F$:

- | | | |
|-----|---|-----------------------------|
| 1. | $I \not\models F$ | assumption |
| 2. | $I \models \text{car}(a) = \text{car}(b)$ | 1, $\rightarrow$, $\wedge$ |
| 3. | $I \models \text{cdr}(a) = \text{cdr}(b)$ | 1, $\rightarrow$, $\wedge$ |
| 4. | $I \models \neg \text{atom}(a)$ | 1, $\rightarrow$, $\wedge$ |
| 5. | $I \models \neg \text{atom}(b)$ | 1, $\rightarrow$, $\wedge$ |
| 6. | $I \not\models f(a) = f(b)$ | 1, $\rightarrow$ |
| 7. | $I \models \text{cons}(\text{car}(a), \text{cdr}(a)) = \text{cons}(\text{car}(b), \text{cdr}(b))$ | 2, 3, (function congruence) |
| 8. | $I \models \text{cons}(\text{car}(a), \text{cdr}(a)) = a$ | 4, (construction) |
| 9. | $I \models \text{cons}(\text{car}(b), \text{cdr}(b)) = b$ | 5, (construction) |
| 10. | $I \models a = b$ | 7, 8, 9, (transitivity) |
| 11. | $I \models f(a) = f(b)$ | 10, (function congruence) |
| 12. | $I \models \perp$ | 6, 11 |

Therefore, F is $T_{\text{cons}}^=$ -valid. ■

3.6 Arrays

Arrays are another common data structure in programming. They are similar to the uninterpreted functions of T_E except that they can be modified. The **theory of arrays** T_A describes the basic characteristic of an array: if value v is written to position i of array a , then subsequently reading from position i of a should return v . Because logic is static, modified arrays are represented functionally, as in functional programming.

The theory of arrays T_A has signature

$$\Sigma_A : \{ \cdot[\cdot], \cdot\langle \cdot \triangleleft \cdot \rangle, = \} ,$$

where

- $a[i]$ (read) is a binary function: $a[i]$ represents the value of array a at position i ;
- $a\langle i \triangleleft v \rangle$ (write) is a ternary function: $a\langle i \triangleleft v \rangle$ represents the modified array a in which position i has value v ;
- $=$ (equality) is a binary predicate.

$\cdot[\cdot]$ and $\cdot\langle\cdot\triangleleft\cdot\rangle$ really are binary and ternary functions, respectively, even though we write them using a convenient notation. Writing $a[i]$ as $\text{read}(a, i)$ and $a\langle i \triangleleft e \rangle$ as $\text{write}(a, i, e)$ emphasizes that they are functions.

Arrays are represented functionally. The term $a\langle i \triangleleft v \rangle$ is an array that is like a except that it has value v at position i . The term $a\langle i \triangleleft v \rangle[j]$ (which abbreviates $(a\langle i \triangleleft v \rangle)[j]$) is equal to the value of array $a\langle i \triangleleft v \rangle$ at position j : it is v if $j = i$ and $a[j]$ otherwise. $a\langle i \triangleleft v \rangle\langle j \triangleleft w \rangle$ (which abbreviates $(a\langle i \triangleleft v \rangle)\langle j \triangleleft w \rangle$) is an array that is like a except that it differs at the positions i , where it has value v , and j , where it has value w . Finally, the term $a\langle i \triangleleft v \rangle\langle j \triangleleft w \rangle[k]$ (which abbreviates $((a\langle i \triangleleft v \rangle)\langle j \triangleleft w \rangle)[k]$) has value w if $k = j$ (even if $k = i$ also), value v if $k = i$ and $k \neq j$, and value $a[k]$ otherwise.

The axioms of T_A are the following:

1. the axioms of (reflexivity), (symmetry), and (transitivity) of T_E ;
2. $\forall a, i, j. i = j \rightarrow a[i] = a[j]$ (array congruence)
3. $\forall a, v, i, j. i = j \rightarrow a\langle i \triangleleft v \rangle[j] = v$ (read-over-write 1)
4. $\forall a, v, i, j. i \neq j \rightarrow a\langle i \triangleleft v \rangle[j] = a[j]$ (read-over-write 2)

The first set of axioms defines $=$ as an equivalence relation. The next axiom asserts that accessing an array with two equal expressions produces the same element. The final two axioms capture the basic characteristic of arrays: reading at an index that has been written produces the most recently written value.

The equality predicate $=$ is only defined for array “elements”. For example,

$$F : a[i] = e \rightarrow a\langle i \triangleleft e \rangle = a$$

is not T_A -valid, although our intuition suggests that it should be. The problem is that the interaction between $=$ and the read and write functions is not captured in the axioms of T_A . In other words, equality between arrays, not just between elements, is undefined.

Instead of F , we write

$$F' : a[i] = e \rightarrow \forall j. a\langle i \triangleleft e \rangle[j] = a[j] ,$$

which is T_A -valid.

Example 3.16. To prove that

$$F' : a[i] = e \rightarrow \forall j. a\langle i \triangleleft e \rangle[j] = a[j] ,$$

is T_A -valid, assume otherwise: there is a T_A -interpretation I such that $I \not\models F'$:

1.	$I \not\models F'$	assumption
2.	$I \models a[i] = e$	1, $\rightarrow$
3.	$I \not\models \forall j. a\langle i \triangleleft e \rangle[j] = a[j]$	1, $\rightarrow$
4.	$I_1 : I \triangleleft \{j \mapsto j\} \not\models a\langle i \triangleleft e \rangle[j] = a[j]$	3, $\forall$, for some $j \in D_I$
5.	$I_1 \models a\langle i \triangleleft e \rangle[j] \neq a[j]$	4, $\neg$
6.	$I_1 \models i = j$	5, (read-over-write 2)
7.	$I_1 \models a[i] = a[j]$	6, (array congruence)
8.	$I_1 \models a\langle i \triangleleft e \rangle[j] = e$	6, (read-over-write 1)
9.	$I_1 \models a\langle i \triangleleft e \rangle[j] = a[j]$	2, 7, 8, (transitivity)
10.	$I_1 \models \perp$	4, 9

We derive line 6 from line 5 by using the **contrapositive** of (read-over-write 2). The contrapositive of $F_1 \rightarrow F_2$ is $\neg F_2 \rightarrow \neg F_1$, and

$$F_1 \rightarrow F_2 \Leftrightarrow \neg F_2 \rightarrow \neg F_1 .$$

Lines 4 and 9 are contradictory, so that actually $I \models F'$. Thus, F' is T_A -valid. ■

Unfortunately, T_A -validity is undecidable. It is straightforward to encode arbitrary formulae of FOL in T_A by viewing functions as multi-dimensional arrays (arrays whose elements are arrays). Therefore, a theory T_A^- in which the behavior of $=$ on arrays is axiomatized has been studied. Its quantifier-free fragment is decidable. The signature of T_A^- is the same as that of T_A . Its axioms consists of those of T_A and the following axiom:

$$\forall a, b. (\forall i. a[i] = b[i]) \leftrightarrow a = b \quad (\text{extensionality})$$

Example 3.17. To prove that

$$F : a[i] = e \rightarrow a\langle i \triangleleft e \rangle = a$$

is T_A^- -valid, assume otherwise: there is a T_A^- -interpretation I such that $I \not\models F$:

1.	$I \not\models F$	assumption
2.	$I \models a[i] = e$	1, $\rightarrow$
3.	$I \not\models a\langle i \triangleleft e \rangle = a$	1, $\rightarrow$
4.	$I \models a\langle i \triangleleft e \rangle \neq a$	3, $\neg$
5.	$I \models \neg(\forall j. a\langle i \triangleleft e \rangle[j] = a[j])$	4, (extensionality)
6.	$I \not\models \forall j. a\langle i \triangleleft e \rangle[j] = a[j]$	5, $\neg$

The rest of the proof then proceeds as in Example 3.16. ■

We present a decision procedure for the quantifier-free fragment of T_A in Chapter 9. In Chapter 11, we present a decision procedure for satisfiability in a fragment of T_A that is more expressive than even the quantifier-free fragment of T_A^- .

Table 3.1. Decidability of theories and quantifier-free fragments

Theory	Description	Full	QFF
T_E	equality	no	yes
T_{PA}	Peano arithmetic	no	no
T_N	Presburger arithmetic	yes	yes
T_Z	linear integers	yes	yes
T_R	reals (with $\cdot$)	yes	yes
T_Q	rational (without $\cdot$)	yes	yes
T_{RDS}	recursive data structures	no	yes
T_{RDS}^+	acyclic recursive data structures	yes	yes
T_A	arrays	no	yes
$T_A^=$	arrays with extensionality	no	yes

Table 3.2. Complexities for decidable theories

Theory	Complexity
PL	NP-complete
T_N, T_Z	$\Omega(2^{2^n}), O(2^{2^{kn}})$
T_R	$O(2^{2^{kn}})$
T_Q	$\Omega(2^n), O(2^{kn})$
T_{RDS}^+	not elementary recursive

3.7 ★Survey of Decidability and Complexity

We survey the known decidability and complexity results of the theories of this chapter.

Table 3.1 summarizes the decidability results for the first-order theories. The quantifier-free fragment of each theory that we study in Part II of this book is decidable.

Table 3.2 summarizes the complexity results for satisfiability in PL and the decidable first-order theories. For all complexities, n is the size of the input formula, and k is some positive integer. A decision problem is not **elementary recursive** if its running time cannot be bounded by a fixed-height stack of exponentials. Only decision procedures for satisfiability in PL scale well to large problems.

Table 3.3 summarizes the complexity results for the quantifier-free fragments. As satisfiability in PL is already NP-complete, we consider only **conjunctive** formulae, which are just conjunctions of literals. For example, satisfiability of propositional conjunctive formulae is decidable in linear time: if both P and $\neg P$ appear in F , for some propositional variable P , then F is unsatisfiable; otherwise, F is satisfiable. For quantifier-free (but not conjunctive) formulae, all complexities except that for T_R are NP-complete. Satisfiability in the quantifier-free fragments of T_E , T_Q , T_{RDS} , and T_{RDS}^+ is efficiently decidable.

Table 3.3. Complexities for quantifier-free, conjunctive fragments of theories

Theory	Complexity	Theory	Complexity
PL	$\Theta(n)$	T_E	$O(n \log n)$
T_N, T_Z	NP-complete	T_R	$O(2^{2^{kn}})$
T_Q	PTIME	T_{RDS}^+	$\Theta(n)$
T_{RDS}	$O(n \log n)$	T_A	NP-complete

3.8 Combination Theories

In practice, the formulae that we want to check for satisfiability or validity span multiple theories. For example, in program verification, one might want to prove a property about an array of integers or a list of reals. We will see many such examples in Chapter 5. Thus, decision procedures for fragments of first-order theories are essentially useless unless they can be combined.

What does every signature of every theory presented so far have in common? They all have equality, $=$. Nelson and Oppen made equality the focal predicate in their general method for combining quantifier-free fragments of first-order theories (with some restrictions). Given two theories T_1 and T_2 such that $\Sigma_1 \cap \Sigma_2 = \{=\}$ — only $=$ is shared — the combined theory $T_1 \cup T_2$ has signature $\Sigma_1 \cup \Sigma_2$ and axioms $A_1 \cup A_2$. Nelson and Oppen showed that if

- satisfiability in the quantifier-free fragment of T_1 is decidable;
- satisfiability in the quantifier-free fragment of T_2 is decidable;
- and certain technical requirements are met,

then satisfiability in the quantifier-free fragment of $T_1 \cup T_2$ is decidable. Furthermore, if the decision procedures for T_1 and T_2 are in P (in NP), then the combined decision procedure for $T_1 \cup T_2$ is in P (in NP).

Chapter 10 studies the Nelson-Oppen combination of decision procedures.

Example 3.18. To prove that the $(\Sigma_A^= \cup \Sigma_Z)$ -formula

$$F : a = b \rightarrow a[i] \geq b[i]$$

is $(T_A^= \cup T_Z)$ -valid we assume otherwise: there is a $(T_A^= \cup T_Z)$ -interpretation I such that $I \not\models F$:

- | | | |
|----|----------------------------------|-----------------------------|
| 1. | $I \not\models F$ | assumption |
| 2. | $I \models a = b$ | 1, $\rightarrow$ |
| 3. | $I \not\models a[i] \geq b[i]$ | 1, $\rightarrow$ |
| 4. | $I \models \neg(a[i] \geq b[i])$ | 3, $\neg$ |
| 5. | $I \models a[i] = b[i]$ | 2, $T_A^=$ (extensionality) |
| 6. | $I \models \perp$ | 4, 5, $T_A^= \cup T_Z$ |

Line 6 summarizes the argument that it is impossible for a T_Z -interpretation to satisfy both $a[i] = b[i]$ and $\neg(a[i] \geq b[i])$. ■

Example 3.19. The $(\Sigma_{\mathbb{E}} \cup \Sigma_{\mathbb{Z}})$ -formula

$$1 \leq x \wedge x \leq 2 \wedge f(x) \neq f(1) \wedge f(x) \neq f(2)$$

is $(T_{\mathbb{E}} \cup T_{\mathbb{Z}})$ -unsatisfiable, for x cannot be either 1 or 2 without violating (function congruence). Seen as a $(\Sigma_{\mathbb{E}} \cup \Sigma_{\mathbb{Q}})$ -formula, it is $(T_{\mathbb{E}} \cup T_{\mathbb{Q}})$ -satisfiable: choose $x = \frac{3}{2}$.

The $(\Sigma_{\mathbb{E}} \cup \Sigma_{\mathbb{Q}})$ -formula

$$f(f(x) - f(y)) \neq f(z) \wedge x \leq y \wedge y + z \leq x \wedge 0 \leq z$$

is $(T_{\mathbb{E}} \cup T_{\mathbb{Q}})$ -unsatisfiable. In particular, the final three literals imply that $z = 0$ and $x = y$, so that $f(x) = f(y)$. But then from the first literal, $f(0) \neq f(0)$ since both $f(x) - f(y)$ and z equal 0.

Finally, the $(\Sigma_{\mathbb{E}} \cup \Sigma_{\mathbb{Z}})$ -formula

$$1 \leq x \wedge x \leq 3 \wedge f(x) \neq f(1) \wedge f(x) \neq f(3) \wedge f(1) \neq f(2)$$

is $(T_{\mathbb{E}} \cup T_{\mathbb{Z}})$ -satisfiable since x can be 2 without violating (function congruence). ■

3.9 Summary

Important data types in software and hardware models include integers; rationals; recursive data structures like records, lists, stacks, and trees; and arrays. This chapter introduces first-order theories that formalize these data types. It covers:

- *First-order theories.* Formalizations of structures and operations into first-order logic: signatures, axioms. Fragments of theories, in particular quantifier-free fragments. Interpretations, satisfiability, validity.
- Specific theories:
 - *Equality* defines the binary predicate $=$ as a congruence relation. Satisfiability in the quantifier-free fragment is efficiently decidable, and the decision procedure is the basis for decision procedures for data structures (see Chapter 9).
 - *Integer arithmetic.* Satisfiability in integer arithmetic without multiplication is decidable.
 - *Rational and real arithmetic.* Satisfiability in real arithmetic with multiplication is decidable with high complexity. Satisfiability in rational arithmetic without multiplication is efficiently decidable. Rational arithmetic without multiplication is indistinguishable from real arithmetic without multiplication.
 - *Recursive data structures* include records, lists, stacks, and queues. Satisfiability in the quantifier-free fragment is efficiently decidable.

- *Arrays* can be read and written. Satisfiability in the quantifier-free fragment is decidable. Chapter 11 studies a larger fragment in which satisfiability is still decidable.
- *Combination theories*. How can decision procedures for multiple theories be combined to decide satisfiability in combination theories?

Studying first-order theories is important for two reasons. First, theories formalize into FOL interesting structures and operations on the structures. Second, satisfiability in some theories or fragments of theories is decidable and thus can be reasoned about algorithmically, whereas satisfiability in general FOL is undecidable. Part II of this book focuses on such theories and fragments that are useful for program analysis. Chapters 5 and 6 provide many examples of formulae from combinations of these theories in the context of program verification.

Bibliographic Remarks

The undecidability of validity in FOL [13] motivated the subsequent study of first-order theories and fragments. In 1929, Presburger proved that satisfiability in arithmetic without multiplication is decidable [73]. Tarski showed in the 1930s that real arithmetic is decidable even with multiplication, although the Second World War delayed the publication of this result [90]. The axiomatization of recursive data structures that we study is from work by Nelson and Oppen [66]. Oppen studied a variation in which structures are acyclic [69]. The axiomatization of arrays, in particular the read-over-write axioms, is due to McCarthy [59]. The Nelson-Oppen combination method is based on work by Nelson and Oppen in the late 1970s and early 1980s [65].

Exercises

3.1 (Semantic argument in T_E). Use the semantic method to argue the validity of the following Σ_E -formulae, or identify a counterexample (a falsifying T_E -interpretation).

- (a) $f(x, y) = f(y, x) \rightarrow f(a, y) = f(y, a)$
- (b) $f(g(x)) = g(f(x)) \wedge f(g(f(y))) = x \wedge f(y) = x \rightarrow g(f(x)) = x$
- (c) $f(f(f(a))) = f(f(a)) \wedge f(f(f(f(a)))) = a \rightarrow f(a) = a$
- (d) $f(f(f(a))) = f(a) \wedge f(f(a)) = a \rightarrow f(a) = a$
- (e) $p(x) \wedge f(f(x)) = x \wedge f(f(f(x))) = x \rightarrow p(f(x))$

3.2 (Semantic argument in $T_{\mathbb{Z}}$). Use the semantic method to argue the validity of the following $\Sigma_{\mathbb{Z}}$ -formulae, or identify a counterexample (a falsifying $T_{\mathbb{Z}}$ -interpretation).

- (a) $x \leq y \wedge z = x + 1 \rightarrow z \leq y$

- (b) $x \leq y \wedge z = x - 1 \rightarrow z \leq y$
- (c) $3x = 2 \rightarrow x \leq 0$
- (d) $1 \leq x \wedge x \leq 2 \rightarrow x = 1 \vee x = 2$
- (e) $1 \leq x \wedge x + y \leq 3 \wedge 1 \leq y \rightarrow x = 1 \vee x = 2$
- (f) $0 \leq x \wedge 0 \leq x + y \wedge x + y \leq 1 \wedge (y \leq -2 \vee 2 \leq y) \rightarrow 0 \leq -1$

3.3 (Semantic argument in $T_{\mathbb{Q}}$). Use the semantic method to argue the validity of the following $\Sigma_{\mathbb{Q}}$ -formulae, or identify a counterexample (a falsifying $T_{\mathbb{Q}}$ -interpretation).

- (a) $3x = 2 \rightarrow x \leq 0$
- (b) $0 \leq x + 2y \wedge 2x + y \leq 1$
- (c) $1 \leq x \wedge x \leq 2 \rightarrow x = 1 \vee x = 2$

3.4 (Semantic argument in T_{cons}). Use the semantic method to argue the validity of the following Σ_{cons} -formulae, or identify a counterexample (a falsifying T_{cons} -interpretation).

- (a) $\text{car}(x) = y \wedge \text{cdr}(x) = z \rightarrow x = \text{cons}(y, z)$
- (b) $\neg \text{atom}(x) \wedge \text{car}(x) = y \wedge \text{cdr}(x) = z \rightarrow x = \text{cons}(y, z)$

3.5 (Semantic argument in T_A). Use the semantic method to argue the validity of the following Σ_A -formulae, or identify a counterexample (a falsifying T_A -interpretation).

- (a) $a\langle i \triangleleft e \rangle[j] = e \rightarrow i = j$
- (b) $a\langle i \triangleleft e \rangle[j] = e \rightarrow a[j] = e$
- (c) $a\langle i \triangleleft e \rangle[j] = e \rightarrow i = j \vee a[j] = e$
- (d) $a\langle i \triangleleft e \rangle\langle j \triangleleft f \rangle[k] = g \wedge j \neq k \wedge i = j \rightarrow a[k] = g$

3.6 (Semantic argument in combinations). For each of the following formulae, identify the combination of theories in which it lies. To avoid ambiguity, prefer $T_{\mathbb{Z}}$ to $T_{\mathbb{Q}}$. Then argue its validity in that combination of theories using the semantic method, or identify a counterexample.

- (a) $1 \leq x \wedge x \leq 2 \wedge \text{cons}(1, y) \neq \text{cons}(x, y) \rightarrow \text{cons}(2, y) = \text{cons}(x, y)$
- (b) $a[i] \geq 1 \wedge a[i] + x \leq 2 \wedge x > 0 \wedge x = i \rightarrow a\langle x \triangleleft 2 \rangle[i] = 1$
- (c) $1 \leq x \wedge x \leq 2 \wedge \text{cons}(1, y) \neq \text{cons}(x, y) \rightarrow \text{cons}(2, y) = \text{cons}(x, y)$
- (d) $x + y = z \wedge f(z) = z \rightarrow f(x + y) = z$
- (e) $g(x + y, z) = f(g(x, y)) \wedge x + z = y \wedge z \geq 0 \wedge x \geq y \wedge g(x, x) = z \rightarrow f(z) = g(2x, 0)$

3.7 (Semantic argument in combinations). Redo Exercise 3.6, preferring $T_{\mathbb{Q}}$ to $T_{\mathbb{Z}}$.